

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I [Mohammad Fuzail / 192565050](#) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 – Intelligent Agents in a Smart Home Automation System

A smart home system manages lighting, temperature and security using motion detection, temperature changes and time of day, and learns user preferences over time. Explain simple reflex, model-based, goal-based and utility-based agents, analyse how each functions in this system, and justify the most effective type or hybrid approach for comfort, energy efficiency and security.

Introduction

An **intelligent agent** perceives through sensors and acts through actuators to maximise a performance measure. Here: sensors = motion, temperature, door/window sensors, cameras, clock; actuators = lights, AC / heater, locks, alarm; performance = comfort, low energy use, security.

Types of agents and how each works in the smart home

Agent type	How it works	In the smart home	Limitation
Simple reflex	Acts only on the current percept using condition–action rules (IF condition THEN action). No memory.	IF motion detected AND time = night THEN light ON. IF temperature > 28°C THEN AC ON. IF door opened while owner away THEN sound alarm.	No memory or history; cannot tell whether the room was just left, fails if the sensor input is incomplete, no learning, no optimisation.
Model-based reflex	Keeps an internal state / model of how the world evolves and how its actions change it, so it can handle partial observability.	Remembers that a person is still in the room even when motion stops (e.g. reading), tracks which rooms are occupied, whether windows are open and the outside temperature trend, so lights are not switched off on a sitting person.	Still follows fixed rules; has no explicit goals or preferences.
Goal-based	Chooses actions by considering the future and whether they lead to a stated goal (uses search / planning).	Goal: “room at 24°C by 7 pm when the owner returns” – the agent plans when to start the AC. Goal: “house secure at night” – plans to lock doors, arm sensors and switch on perimeter lights.	Only separates goal achieved / not achieved; cannot judge which of several ways to reach the goal is better (comfort vs. cost).
Utility-based	Uses a utility function that measures how desirable a state is and picks the action with the highest expected utility; handles trade-offs and uncertainty.	Utility = $w_1 \cdot \text{comfort} - w_2 \cdot \text{energy cost} + w_3 \cdot \text{security}$. It decides, for example, to pre-cool slightly less to save energy during peak tariff, or to keep a night light on at low brightness (a good balance of comfort and power).	Needs a good utility function and more computation; the weights must match user preference.
Learning agent (add-on)	A learning element improves the agent's rules / utility from experience (feedback, user overrides).	Learns that the owner prefers 22°C in the evening and dim lights after 10 pm, and adjusts automatically.	Needs data and time to learn.

Analysis for the three goals

- **Comfort:** needs memory of occupancy and learned preferences – model-based + learning agent.
- **Energy efficiency:** needs a comfort vs. cost trade-off – utility-based agent with goal-based planning.
- **Security:** needs immediate response to intrusion – simple reflex rules, which override everything else.

Justification: a hybrid (layered) agent is the most effective

No single type satisfies all requirements, so the best design is a **hybrid agent** with three layers:

- **Reflex layer (highest priority):** instant safety / security rules – intrusion alarm, smoke alert. Fast, no planning delay.
- **Model-based + goal-based layer (middle):** maintains the state of the house (occupancy, open windows, temperature trend, time) and plans to reach goals such as “comfortable on arrival” and “secure at night”.
- **Utility-based + learning layer (top):** chooses among alternative plans by maximising utility (comfort vs. energy cost vs. security) and learns user habits from feedback so utility weights improve over time.

Conclusion: the hybrid agent reacts instantly (reflex), understands the house (model-based), plans (goal-based), balances comfort and energy (utility-based) and personalises itself (learning).

Question 2 – Robot in an Unknown Maze: UCS, IDS and Agent Design

A robot is placed in an unknown maze and must find the exit with no prior knowledge. Explain how UCS and IDS help; discuss advantages and disadvantages (time and space complexity); relate the problem to intelligent agent types and how design affects decisions; suggest and justify the best combination of agent type and search strategy.

Problem formulation

States: robot position in the maze. **Actions:** move to a free neighbouring cell. **Initial state:** start cell. **Goal test:** robot is at the exit. **Path cost:** cost of each move (distance / time / energy). Because the layout is unknown, only *uninformed* (blind) strategies that need no heuristic are applicable.

Uniform Cost Search (UCS)

UCS keeps a priority queue ordered by the path cost $g(n)$ and always expands the cheapest unexpanded node. If moves have different costs (narrow corridors, slopes, risky areas), UCS finds the **least-cost path** to the exit. With equal step cost it behaves like BFS.

- **Advantages:** complete (if step cost $\geq \text{eps} > 0$) and optimal; handles unequal step costs; guarantees the cheapest route.
- **Disadvantages:** explores in all directions with no sense of where the exit is; may expand very many nodes; needs to store the whole frontier.
- **Complexity:** with C^* = optimal cost and eps = minimum step cost, time and space are both $O(b^{1 + C^*/\text{eps}})$ – exponential. Space is the main practical problem because the frontier and explored set are kept in memory.

Iterative Deepening Search (IDS)

IDS runs depth-limited DFS with limits 0, 1, 2, 3, ... until the exit is found. It combines the small memory use of DFS with the completeness and shallowest-solution property of BFS. Nodes at upper levels are re-generated many times, but this repetition is small because most nodes lie at the deepest level.

- **Advantages:** very low memory; complete (finite branching); optimal when every move costs the same (finds the shallowest exit); suited to a robot with limited memory; no need to know the depth of the exit in advance.
- **Disadvantages:** repeats work at shallow levels; not optimal when step costs differ; may re-visit cells unless loops are checked.
- **Complexity:** time $O(b^d)$ (d = depth of the exit, b = branching factor – at most 3 in a maze, so small); space only $O(b \cdot d)$ (linear).

Comparison

Property	UCS	IDS
Complete	Yes (step cost $\geq \text{eps}$)	Yes (finite b)
Optimal	Yes – least cost, any step costs	Yes only for equal step costs (fewest moves)
Time	$O(b^{1+C^*/\text{eps}})$	$O(b^d)$
Space	$O(b^{1+C^*/\text{eps}})$ – high	$O(b \cdot d)$ – very low
Best used when	Moves have different costs	Memory is limited, costs are equal

Relation to intelligent agent types and effect of agent design

- **Simple reflex agent** (e.g. “turn left at a wall”): has no memory, so it can loop forever in a maze and cannot use search – unsuitable.
- **Model-based agent:** builds a map of visited cells and walls from its sensors. This is essential in an unknown maze (partially observable), because search needs a stored state to avoid repeated cells and to back up from dead ends.
- **Goal-based agent:** its goal is “reach the exit”; it plans a sequence of moves using UCS or IDS on its map. Search is the tool of a goal-based agent.
- **Utility-based agent:** adds preferences – lowest energy, avoid hazards, shortest time – and chooses the best of several routes; corresponds to UCS with realistic costs.
- **Effect of design:** the agent's sensing and memory decide what it can know, and so what decisions it can make. A reflex design decides on the current cell only (poor, may loop); a model-based design remembers the maze (avoids loops); a goal/utility design looks ahead and compares routes (rational, optimal decisions).

Best combination and justification

Recommended: a model-based, goal-based (utility-aware) agent using Iterative Deepening Search, switching to UCS when step costs differ.

- **Model-based:** the maze is unknown and only nearby walls are sensed, so the robot must remember the map it discovers (online exploration, updating the map as it moves).
- **Goal-based / utility-aware:** the single clear goal is the exit, and cost (energy / time) is the quantity to minimise.
- **IDS:** memory is limited on a robot, the maze has a small branching factor (≤ 3), and with unit moves IDS finds the shortest path using only $O(b \cdot d)$ memory with total time of the same order as BFS.
- **UCS:** used instead when parts of the maze have different costs (slopes, hazards) so that the least-cost path is guaranteed, if the memory permits.
- **Conclusion:** this combination is complete and optimal, memory-efficient, and suited to an unknown, partially observable maze; its decisions are rational because the robot remembers where it has been and plans toward its goal.